Pack 16 (Desktop) — Session Approval Consent + Clipboard/File-Transfer Gating + Session Expiry Enforcement (real code)

RemoteDesk · Pack 16 (Desktop) — Session Approval Consent + Clipboard/File-Transfer Gating + Session Expiry Enforcement (real code)

> Codex / Claude Code handoff. Direct continuation of Desktop Pack 15 (device policy enforcement). This slice wires the three things Pack 15 left open: the `requiresSessionApproval` consent flow, real clipboard + file-transfer gating on the data channel, and a hard session-expiry cutoff. Build order: shared additions → consent state → channel gates → expiry watcher → UI → tests.
> Scope: enforce host consent before a session goes interactive, gate clipboard/file-transfer frames on the live permission set, and hard-stop input (not the screen view) at expiry. **Fail-closed everywhere. Remote input still off by default. Host stays the source of truth. No native input executor. No secrets in logs.**
> Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

```
CREATED [NEW]
  packages/shared/src/permissions/sessionApproval.ts
  apps/desktop/src/renderer/src/services/sessionApprovalState.ts
  apps/desktop/src/renderer/src/services/channelFeatureGate.ts
  apps/desktop/src/renderer/src/services/sessionExpiryWatcher.ts
  apps/desktop/src/renderer/src/components/SessionApprovalPrompt.tsx
  apps/desktop/src/renderer/src/components/SessionExpiryBadge.tsx
  tests/permissions/sessionApproval.test.ts
  tests/permissions/channelFeatureGate.test.ts
  tests/permissions/sessionExpiry.test.ts

MODIFIED [MODIFY]
  packages/shared/src/permissions/index.ts                      (export sessionApproval)
  apps/desktop/src/renderer/src/services/sessionDataChannel.ts   (consent frame + gate clipboard/file)
  apps/desktop/src/renderer/src/services/permissionState.ts      (consent gate before accept)
  apps/desktop/src/renderer/src/components/RemoteSessionView.tsx  (approval prompt + expiry badge)

NOT TOUCHED (intentionally)
  apps/desktop/src/main/index.ts          (no native executor)
  apps/desktop/src/preload/index.ts       (no new bridge)
  apps/desktop/src/renderer/src/services/socket.ts
  apps/desktop/src/renderer/src/services/webrtc.ts (input gate from Pack 15 unchanged)
  apps/api/**  apps/web/**                (consent is session-local; policy already persisted)
```

Part A — Shared: session approval contract

FILE: packages/shared/src/permissions/sessionApproval.ts

```
import { allow, deny, type DeviceAccessPolicySnapshot, type PolicyDecision } from './devicePolicy';

/**
 * Session approval (consent) state. When a policy has requiresSessionApproval,
 * the host must explicitly approve before ANY interactive feature (remote input,
 * clipboard, file transfer) can be enabled — screen viewing is unaffected.
 */
export type SessionApprovalStatus = 'pending' | 'approved' | 'denied' | 'not_required';

export interface SessionApprovalSnapshot {
  status: SessionApprovalStatus;
  /** ISO; when the decision was made (null while pending/not_required). */
  decidedAt: string | null;
  /** Optional viewer identity for the prompt (display-only, non-sensitive). */
  requestedByLabel: string | null;
}

/** Initial approval state derived from the policy. */
export function initialApproval(
  policy: DeviceAccessPolicySnapshot | null,
  requestedByLabel: string | null = null,
): SessionApprovalSnapshot {
  // Fail closed: if policy is missing we still require explicit approval.
  const required = policy ? policy.requiresSessionApproval : true;
  return {
    status: required ? 'pending' : 'not_required',
    decidedAt: null,
    requestedByLabel,
  };
}

/**
 * Is host acceptance permitted to take effect yet? Interactive features must wait
 * until approval is 'approved' or 'not_required'. Pending/denied => not effective.
 */
export function isApprovalSatisfied(approval: SessionApprovalSnapshot): boolean {
  return approval.status === 'approved' || approval.status === 'not_required';
}

/**
 * Gate "host accepted" through the approval state. Even if the host toggled
 * "allow control", it only counts once consent is satisfied. This is the bridge
 * between Pack 15's hostAccepted flag and the consent requirement.
 */
export function effectiveHostAccepted(
  hostAccepted: boolean,
  approval: SessionApprovalSnapshot,
): boolean {
  return hostAccepted && isApprovalSatisfied(approval);
}

/** Pure transition guard for the approval lifecycle. */
export function canDecide(approval: SessionApprovalSnapshot): boolean {
```

```
  return approval.status === 'pending';
}

/** Decide consent. Returns a new snapshot; throws nothing (callers guard with canDecide). */
export function decideApproval(
  approval: SessionApprovalSnapshot,
  decision: 'approved' | 'denied',
  now: Date = new Date(),
): SessionApprovalSnapshot {
  if (!canDecide(approval)) return approval;
  return { ...approval, status: decision, decidedAt: now.toISOString() };
}

/** Decision-style helper for callers that want a PolicyDecision shape. */
export function evaluateApprovalGate(approval: SessionApprovalSnapshot): PolicyDecision {
  return isApprovalSatisfied(approval) ? allow() : deny('host_not_accepted');
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval'; // ADD
```

Part B — Desktop: session approval state

FILE: apps/desktop/src/renderer/src/services/sessionApprovalState.ts

```
import {
  initialApproval,
  decideApproval,
  type DeviceAccessPolicySnapshot,
  type SessionApprovalSnapshot,
} from '@remotedesk/shared';

/**
 * Host-side session approval state. Owns the consent lifecycle for a session and
 * notifies subscribers (UI + permission state) on change. Viewer never owns this;
 * it only sees the resulting permission snapshot from Pack 15's broadcast.
 */
export class SessionApprovalState {
  private approval: SessionApprovalSnapshot;
  private listeners = new Set<(a: SessionApprovalSnapshot) => void>();

  constructor(policy: DeviceAccessPolicySnapshot | null, requestedByLabel: string | null = null) {
    this.approval = initialApproval(policy, requestedByLabel);
  }

  current(): SessionApprovalSnapshot {
    return this.approval;
  }

  approve(): void {
    this.approval = decideApproval(this.approval, 'approved');
    this.emit();
```

```
  }

  deny(): void {
    this.approval = decideApproval(this.approval, 'denied');
    this.emit();
  }

  /** Re-derive when the policy changes mid-session (e.g. requiresSessionApproval flips). */
  resetFromPolicy(policy: DeviceAccessPolicySnapshot | null, requestedByLabel: string | null = null): void {
    this.approval = initialApproval(policy, requestedByLabel);
    this.emit();
  }

  subscribe(fn: (a: SessionApprovalSnapshot) => void): () => void {
    this.listeners.add(fn);
    return () => this.listeners.delete(fn);
  }

  private emit(): void {
    for (const fn of this.listeners) fn(this.approval);
  }
}
```

FILE: apps/desktop/src/renderer/src/services/permissionState.ts [MODIFY]

```
// ...existing imports from Pack 15...
import {
  effectiveHostAccepted,
  type SessionApprovalSnapshot,
} from '@remotedesk/shared';

// ---- ADD: hold the current approval snapshot in HostPermissionState ----
// private approval: SessionApprovalSnapshot = { status: 'pending', decidedAt: null, requestedByLabel: null };
//
// setApproval(approval: SessionApprovalSnapshot): void {
//   this.approval = approval;
//   this.emit();
// }

// ---- MODIFY: where current() builds the permission set, pass the CONSENT-GATED
// hostAccepted instead of the raw flag, so interactive features stay off until
// the host approves the session: ----
//
//   const gatedAccepted = effectiveHostAccepted(this.hostAccepted, this.approval);
//   return buildSessionPermissionSet({
//     policy: this.policy,
//     hostAccepted: gatedAccepted,   // <-- was this.hostAccepted
//     emergencyStopped: this.emergencyStopped,
//     startedAt: this.startedAt,
//     version: this.version,
//   });
```

Part C — Desktop: clipboard + file-transfer channel gate

FILE: apps/desktop/src/renderer/src/services/channelFeatureGate.ts

```
import type { SessionPermissionSet } from '@remotedesk/shared';

/**
 * Gates clipboard-sync and file-transfer frames on the live permission set.
 * Mirrors the remote-input gate from Pack 15 but for the other two interactive
 * features. Fail closed: no permission snapshot => everything denied. Blocked
 * frames are counted as non-sensitive diagnostics (no payload contents).
 */
export class ChannelFeatureGate {
  private permissions: SessionPermissionSet | null = null;
  private blocked = { clipboard: 0, fileTransfer: 0 };

  update(permissions: SessionPermissionSet | null): void {
    this.permissions = permissions;
  }

  private active(now: Date): boolean {
    const p = this.permissions;
    if (!p) return false;
    if (p.emergencyStopped) return false;
    if (p.sessionExpiresAt && now.getTime() >= Date.parse(p.sessionExpiresAt)) return false;
    return true;
  }

  clipboardAllowed(now: Date = new Date()): boolean {
    return this.active(now) && !!this.permissions?.clipboardSync;
  }

  fileTransferAllowed(now: Date = new Date()): boolean {
    return this.active(now) && !!this.permissions?.fileTransfer;
  }

  /** Call before sending/applying a clipboard frame. */
  guardClipboard(): boolean {
    if (this.clipboardAllowed()) return true;
    this.record('clipboard');
    return false;
  }

  /** Call before sending/accepting a file-transfer request. */
  guardFileTransfer(): boolean {
    if (this.fileTransferAllowed()) return true;
    this.record('fileTransfer');
    return false;
  }

  blockedTotals(): { clipboard: number; fileTransfer: number } {
    return { ...this.blocked };
  }

  private record(feature: 'clipboard' | 'fileTransfer'): void {
    this.blocked[feature] += 1;
    const count = this.blocked[feature];
    if (count === 1 || count % 50 === 0) {
      console.info('[channel-gate] blocked frame', {
```

```
        feature,
        emergencyStopped: this.permissions?.emergencyStopped ?? false,
        blockedTotal: count,
      });
    }
  }
}
```

FILE: apps/desktop/src/renderer/src/services/sessionDataChannel.ts [MODIFY]

```
// ...existing imports + Pack 15 permission frame helpers...
import type { SessionApprovalSnapshot } from '@remotedesk/shared';
import { ChannelFeatureGate } from './channelFeatureGate';

// ---- ADD: shared channel feature gate for this session ----
export const channelFeatureGate = new ChannelFeatureGate();

// Call this wherever you apply a permission snapshot (host computes / viewer receives):
//   channelFeatureGate.update(permissions);

// ---- ADD: optional consent-request frame so the viewer can show "waiting" ----
export interface ConsentStatusFrame {
  type: 'consent.status';
  payload: SessionApprovalSnapshot;
}
// Merge ConsentStatusFrame into your DataChannelFrame union.

// ---- MODIFY: gate the existing clipboard send/apply path ----
// export function sendClipboard(data: ClipboardPayload): void {
//   if (!channelFeatureGate.guardClipboard()) return; // denied by policy/consent/expiry
//   clipboardChannel.send(serializeClipboard(data));  // existing behavior
// }
// clipboardChannel.on('clipboard', (frame) => {
//   if (!channelFeatureGate.guardClipboard()) return;
//   applyClipboard(frame);                            // existing behavior
// });

// ---- MODIFY: gate the existing file-transfer request path ----
// fileTransfer.onTransferRequest((req) => {
//   if (!channelFeatureGate.guardFileTransfer()) { fileTransfer.reject(req.id, 'blocked_by_policy'); return; }
//   fileTransfer.accept(req.id);                      // existing behavior
// });
```

Part D — Desktop: session expiry watcher

FILE: apps/desktop/src/renderer/src/services/sessionExpiryWatcher.ts

```
import { isSessionExpired, type SessionPermissionSet } from '@remotedesk/shared';

/**
 * Watches the live permission set for session expiry. On expiry it fires a
 * callback ONCE so the session can hard-stop interactive features (input,
 * clipboard, file transfer) WITHOUT disconnecting the screen view. The screen
 * stays up; only control is cut.
 */
```

```
export class SessionExpiryWatcher {
  private timer: ReturnType<typeof setInterval> | null = null;
  private firedFor: string | null = null;

  constructor(
    private getPermissions: () => SessionPermissionSet | null,
    private onExpired: (permissions: SessionPermissionSet) => void,
    private readonly tickMs = 1000,
  ) {}

  start(): void {
    if (this.timer) return;
    this.timer = setInterval(() => this.tick(), this.tickMs);
  }

  stop(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
  }

  /** Exposed for tests + manual checks. */
  tick(now: Date = new Date()): void {
    const p = this.getPermissions();
    if (!p || !p.sessionExpiresAt) return;
    if (this.firedFor === p.sessionExpiresAt) return; // already handled this expiry
    if (isSessionExpired(p.sessionExpiresAt, now)) {
      this.firedFor = p.sessionExpiresAt;
      this.onExpired(p);
    }
  }

  /** Remaining whole seconds until expiry, or null if uncapped/unknown. */
  remainingSeconds(now: Date = new Date()): number | null {
    const p = this.getPermissions();
    if (!p || !p.sessionExpiresAt) return null;
    const ms = Date.parse(p.sessionExpiresAt) - now.getTime();
    return Math.max(0, Math.floor(ms / 1000));
  }
}
```

Part E — UI

FILE: apps/desktop/src/renderer/src/components/SessionApprovalPrompt.tsx

```
import React from 'react';
import type { SessionApprovalSnapshot } from '@remotedesk/shared';

/**
 * Host-only consent prompt. Shown while approval is pending. Approving lets the
 * host's "allow control" take effect; denying keeps the session view-only. Does
 * NOT block screen viewing.
 */
export function SessionApprovalPrompt({
  approval,
  onApprove,
```

```
  onDeny,
}: {
  approval: SessionApprovalSnapshot;
  onApprove: () => void;
  onDeny: () => void;
}) {
  if (approval.status !== 'pending') return null;
  return (
    <div className="session-approval-prompt" role="dialog" aria-modal="false">
      <p className="session-approval-prompt__text">
        {approval.requestedByLabel
          ? `${approval.requestedByLabel} is requesting an interactive session.`
          : 'A viewer is requesting an interactive session.'}
        {' '}Screen viewing is read-only until you approve.
      </p>
      <div className="session-approval-prompt__actions">
        <button type="button" className="btn btn--primary" onClick={onApprove}>Approve control</button>
        <button type="button" className="btn btn--ghost" onClick={onDeny}>Keep view-only</button>
      </div>
    </div>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/SessionExpiryBadge.tsx

```
import React, { useEffect, useState } from 'react';
import type { SessionExpiryWatcher } from '../services/sessionExpiryWatcher';

/**
 * Small countdown badge. Shows remaining session time when the policy caps it.
 * Hidden when uncapped. Purely informational.
 */
export function SessionExpiryBadge({ watcher }: { watcher: SessionExpiryWatcher }) {
  const [remaining, setRemaining] = useState<number | null>(watcher.remainingSeconds());

  useEffect(() => {
    const id = setInterval(() => setRemaining(watcher.remainingSeconds()), 1000);
    return () => clearInterval(id);
  }, [watcher]);

  if (remaining === null) return null;
  const mm = Math.floor(remaining / 60);
  const ss = String(remaining % 60).padStart(2, '0');
  const low = remaining <= 60;

  return (
    <span className={`session-expiry-badge ${low ? 'session-expiry-badge--low' : ''}`}>
      Session ends in {mm}:{ss}
    </span>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```
// ...existing imports + Pack 15 additions...
import { SessionApprovalPrompt } from './SessionApprovalPrompt';
```

```
import { SessionExpiryBadge } from './SessionExpiryBadge';
import { channelFeatureGate } from '../services/sessionDataChannel';
import { SessionExpiryWatcher } from '../services/sessionExpiryWatcher';
import { applyPermissionsToWebRtc } from '../services/webrtc';
import type { SessionApprovalSnapshot } from '@remotedesk/shared';

// ---- ADD: construct once at session start (host) ----
// const approvalState = new SessionApprovalState(policy, viewerLabel);
// const expiryWatcher = new SessionExpiryWatcher(
//   () => permissions,                      // your current permission set getter
//   (p) => {                                // on expiry: cut control, keep video
//     // host: flip accepted off + rebroadcast; viewer: gates already deny via expiry
//     hostPermissionState.setHostAccepted(false);
//   },
// );
// useEffect(() => { expiryWatcher.start(); return () => expiryWatcher.stop(); }, []);

// ---- ADD: keep both gates in sync on every permission change ----
// useEffect(() => {
//   applyPermissionsToWebRtc(permissions);   // Pack 15 input gate
//   channelFeatureGate.update(permissions);  // this pack: clipboard/file gate
// }, [permissions]);

// ---- ADD: feed approval into the host permission state ----
// useEffect(() => approvalState.subscribe((a: SessionApprovalSnapshot) => {
//   hostPermissionState.setApproval(a);      // re-derives + rebroadcasts permissions
// }), []);

// ---- ADD to render (host only): consent prompt + expiry badge ----
// {isHost && (
//   <SessionApprovalPrompt
//     approval={approvalState.current()}
//     onApprove={() => approvalState.approve()}
//     onDeny={() => approvalState.deny()}
//   />
// )}
// <SessionExpiryBadge watcher={expiryWatcher} />
//
// Existing PolicyStatusBanner + EmergencyStopButton + read-only video stay as-is.
```

Part F — Tests

FILE: tests/permissions/sessionApproval.test.ts

```
import { describe, it, expect } from 'vitest';
import {
  initialApproval,
  isApprovalSatisfied,
  effectiveHostAccepted,
  decideApproval,
  canDecide,
  evaluateApprovalGate,
  type DeviceAccessPolicySnapshot,
} from '@remotedesk/shared';
```

```
function policy(over: Partial<DeviceAccessPolicySnapshot> = {}): DeviceAccessPolicySnapshot {
  return {
    deviceId: 'dev-1', trustStatus: 'trusted', unattendedAccessEnabled: false,
    remoteInputEnabled: true, clipboardSyncEnabled: true, fileTransferEnabled: true,
    requiresSessionApproval: true, maxSessionMinutes: null, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('sessionApproval', () => {
  it('starts pending when policy requires approval', () => {
    expect(initialApproval(policy()).status).toBe('pending');
  });

  it('is not_required when policy does not require approval', () => {
    expect(initialApproval(policy({ requiresSessionApproval: false })).status).toBe('not_required');
  });

  it('fails closed: missing policy still requires approval', () => {
    expect(initialApproval(null).status).toBe('pending');
  });

  it('host acceptance is ineffective until approved', () => {
    const pending = initialApproval(policy());
    expect(effectiveHostAccepted(true, pending)).toBe(false);
    const approved = decideApproval(pending, 'approved');
    expect(effectiveHostAccepted(true, approved)).toBe(true);
  });

  it('denied keeps interactive features off', () => {
    const denied = decideApproval(initialApproval(policy()), 'denied');
    expect(isApprovalSatisfied(denied)).toBe(false);
    expect(effectiveHostAccepted(true, denied)).toBe(false);
  });

  it('only pending can be decided', () => {
    const approved = decideApproval(initialApproval(policy()), 'approved');
    expect(canDecide(approved)).toBe(false);
    expect(decideApproval(approved, 'denied').status).toBe('approved'); // no-op
  });

  it('evaluateApprovalGate maps to a PolicyDecision', () => {
    expect(evaluateApprovalGate(initialApproval(policy())).reason).toBe('host_not_accepted');
    expect(evaluateApprovalGate(initialApproval(policy({ requiresSessionApproval: false })))).allowed).toBe(true);
  });
});
```

FILE: tests/permissions/channelFeatureGate.test.ts

```
import { describe, it, expect } from 'vitest';
import { ChannelFeatureGate } from '../../apps/desktop/src/renderer/src/services/channelFeatureGate';
import type { SessionPermissionSet } from '@remotedesk/shared';

function perms(over: Partial<SessionPermissionSet> = {}): SessionPermissionSet {
  return {
    deviceId: 'dev-1', remoteInput: 'enabled', clipboardSync: true, fileTransfer: true,
```

```
    sessionExpiresAt: null, emergencyStopped: false, version: 1, updatedAt: new Date().toISOString(),
    ...over,
  };
}

describe('ChannelFeatureGate', () => {
  it('fails closed with no permissions', () => {
    const g = new ChannelFeatureGate();
    expect(g.clipboardAllowed()).toBe(false);
    expect(g.fileTransferAllowed()).toBe(false);
  });

  it('allows clipboard + file transfer when permitted', () => {
    const g = new ChannelFeatureGate();
    g.update(perms());
    expect(g.guardClipboard()).toBe(true);
    expect(g.guardFileTransfer()).toBe(true);
  });

  it('denies each feature when its flag is off', () => {
    const g = new ChannelFeatureGate();
    g.update(perms({ clipboardSync: false }));
    expect(g.clipboardAllowed()).toBe(false);
    g.update(perms({ fileTransfer: false }));
    expect(g.fileTransferAllowed()).toBe(false);
  });

  it('emergency stop denies both', () => {
    const g = new ChannelFeatureGate();
    g.update(perms({ emergencyStopped: true }));
    expect(g.clipboardAllowed()).toBe(false);
    expect(g.fileTransferAllowed()).toBe(false);
  });

  it('expiry denies both', () => {
    const g = new ChannelFeatureGate();
    g.update(perms({ sessionExpiresAt: '2026-06-15T10:00:00.000Z' }));
    expect(g.clipboardAllowed(new Date('2026-06-15T10:00:01.000Z'))).toBe(false);
  });

  it('counts blocked frames', () => {
    const g = new ChannelFeatureGate();
    g.guardClipboard();
    g.guardFileTransfer();
    expect(g.blockedTotals()).toEqual({ clipboard: 1, fileTransfer: 1 });
  });
});
```

FILE: tests/permissions/sessionExpiry.test.ts

```
import { describe, it, expect, vi } from 'vitest';
import { SessionExpiryWatcher } from '../../apps/desktop/src/renderer/src/services/sessionExpiryWatcher';
import type { SessionPermissionSet } from '@remotedesk/shared';

function perms(expiresAt: string | null): SessionPermissionSet {
  return {
```

```
    deviceId: 'dev-1', remoteInput: 'enabled', clipboardSync: true, fileTransfer: true,
    sessionExpiresAt: expiresAt, emergencyStopped: false, version: 1, updatedAt: new Date().toISOString(),
  };
}

describe('SessionExpiryWatcher', () => {
  it('does not fire when uncapped', () => {
    const onExpired = vi.fn();
    const w = new SessionExpiryWatcher(() => perms(null), onExpired);
    w.tick(new Date('2030-01-01T00:00:00.000Z'));
    expect(onExpired).not.toHaveBeenCalled();
  });

  it('fires once at/after expiry', () => {
    const onExpired = vi.fn();
    const expiry = '2026-06-15T10:30:00.000Z';
    const w = new SessionExpiryWatcher(() => perms(expiry), onExpired);
    w.tick(new Date('2026-06-15T10:29:59.000Z'));
    expect(onExpired).not.toHaveBeenCalled();
    w.tick(new Date('2026-06-15T10:30:00.000Z'));
    w.tick(new Date('2026-06-15T10:30:05.000Z'));
    expect(onExpired).toHaveBeenCalledTimes(1); // fired only once for this expiry
  });

  it('computes remaining seconds', () => {
    const expiry = '2026-06-15T10:30:00.000Z';
    const w = new SessionExpiryWatcher(() => perms(expiry), vi.fn());
    expect(w.remainingSeconds(new Date('2026-06-15T10:29:00.000Z'))).toBe(60);
    expect(w.remainingSeconds(new Date('2026-06-15T10:31:00.000Z'))).toBe(0);
    expect(new SessionExpiryWatcher(() => perms(null), vi.fn()).remainingSeconds()).toBeNull();
  });
});
```

Part G — Reports

FILE: generated-pack16-desktop-consent-gating-merge-summary.md

```
# Pack 16 (Desktop) — Consent + Clipboard/File Gating + Expiry · Merge Summary

## What this slice does
Continues Desktop Pack 15. Adds: (1) a session-approval consent flow so
requiresSessionApproval is actually enforced before any interactive feature, (2)
real clipboard + file-transfer gating on the data channel mirroring the input
gate, and (3) a hard session-expiry cutoff that kills control but keeps the
screen view. Fail-closed throughout; host stays the source of truth.

## Files created (9)
- packages/shared/src/permissions/sessionApproval.ts
- apps/desktop/src/renderer/src/services/sessionApprovalState.ts
- apps/desktop/src/renderer/src/services/channelFeatureGate.ts
- apps/desktop/src/renderer/src/services/sessionExpiryWatcher.ts
- apps/desktop/src/renderer/src/components/SessionApprovalPrompt.tsx
- apps/desktop/src/renderer/src/components/SessionExpiryBadge.tsx
- tests/permissions/sessionApproval.test.ts
- tests/permissions/channelFeatureGate.test.ts
```

```
- tests/permissions/sessionExpiry.test.ts

## Files modified (4)
- packages/shared/src/permissions/index.ts — export sessionApproval.
- apps/desktop/src/renderer/src/services/sessionDataChannel.ts — channel gate + consent frame + clipboard/file guards.
- apps/desktop/src/renderer/src/services/permissionState.ts — gate hostAccepted through consent (effectiveHostAccepted).
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — approval prompt + expiry badge + gate sync.

## Files intentionally NOT touched
- apps/desktop/src/main/index.ts, preload/index.ts — no native executor / no new bridge.
- apps/desktop/src/renderer/src/services/webrtc.ts — Pack 15 input gate unchanged.
- apps/api/**, apps/web/** — consent is session-local; policy already persisted.

## Remaining manual wiring risks
1. Construct SessionApprovalState + SessionExpiryWatcher at session start (host) and
   subscribe approval -> hostPermissionState.setApproval(...).
2. In permissionState.current(), pass effectiveHostAccepted(hostAccepted, approval).
3. Merge consent.status frame into the DataChannelFrame union (optional viewer "waiting" UI).
4. Wrap existing clipboard send/apply + file-transfer request paths with the gate guards.
5. On expiry callback, flip host acceptance off (gates already deny via expiry too).

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: requiresSessionApproval=true => input/clipboard/file stay off until host approves;
  at maxSessionMinutes the badge hits 0:00, control cuts, video keeps streaming.
```

FILE: generated-pack16-desktop-consent-gating-code-review.md

```
# Pack 16 (Desktop) — Code Review Notes

## Strengths
- Consent is fail-closed: missing policy still requires approval; pending/denied keep features off.
- effectiveHostAccepted cleanly bridges Pack 15's hostAccepted flag with the consent requirement — one place, no drift.
- Clipboard + file gate mirrors the input gate (emergency stop + expiry honored in all three).
- Expiry watcher fires once per expiry and cuts control without touching the screen view.
- Pure logic (approval + expiry math) fully unit-tested; gate tested in isolation.

## Reviewer checklist
- [ ] permissionState.current() uses effectiveHostAccepted(hostAccepted, approval).
- [ ] Clipboard/file send + receive paths wrapped with guardClipboard/guardFileTransfer.
- [ ] consent.status merged into DataChannelFrame union (if showing viewer waiting state).
- [ ] Expiry callback flips host acceptance off; confirm rebroadcast to viewer.
- [ ] No payload contents logged by the channel gate (counts only — verified).

## Safety
- Interactive features OFF until trusted + policy-enabled + host-accepted + consent-satisfied.
- Native OS execution remains disabled/no-op.
- Screen viewing is never interrupted by consent, gating, or expiry.
```

FILE: generated-pack16-desktop-consent-gating-manifest.json

```
{
  "pack": "pack16-desktop-consent-gating",
  "title": "Session Approval Consent + Clipboard/File-Transfer Gating + Session Expiry Enforcement",
  "actualFileCount": 13,
  "filesCreated": [
```

```
      "packages/shared/src/permissions/sessionApproval.ts",
      "apps/desktop/src/renderer/src/services/sessionApprovalState.ts",
      "apps/desktop/src/renderer/src/services/channelFeatureGate.ts",
      "apps/desktop/src/renderer/src/services/sessionExpiryWatcher.ts",
      "apps/desktop/src/renderer/src/components/SessionApprovalPrompt.tsx",
      "apps/desktop/src/renderer/src/components/SessionExpiryBadge.tsx",
      "tests/permissions/sessionApproval.test.ts",
      "tests/permissions/channelFeatureGate.test.ts",
      "tests/permissions/sessionExpiry.test.ts"
    ],
    "filesModified": [
      "packages/shared/src/permissions/index.ts",
      "apps/desktop/src/renderer/src/services/sessionDataChannel.ts",
      "apps/desktop/src/renderer/src/services/permissionState.ts",
      "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
    ],
    "filesNotTouched": [
      "apps/desktop/src/main/index.ts",
      "apps/desktop/src/preload/index.ts",
      "apps/desktop/src/renderer/src/services/socket.ts",
      "apps/desktop/src/renderer/src/services/webrtc.ts",
      "apps/api/**",
      "apps/web/**"
    ],
    "dependenciesRequired": {
      "new": [],
      "existing": ["@remotedesk/shared", "react", "vitest"],
      "assumedFromProject": [
        "Desktop Pack 15 permission enforcement (HostPermissionState, remoteInputGate, permission.snapshot frame)",
        "existing clipboard + file-transfer data channel paths",
        "existing RemoteSessionView host/viewer role flag"
      ]
    },
    "knownLimitations": [
      "Consent is session-local (not persisted server-side); re-prompts each session.",
      "Expiry cutoff cuts control only; screen view continues until user disconnects.",
      "Native OS input execution remains disabled/no-op.",
      "Viewer waiting-state UI is optional (depends on merging consent.status frame)."
    ],
    "remainingManualWiringRisks": [
      "Wire effectiveHostAccepted into permissionState.current().",
      "Wrap clipboard/file send+receive with the gate guards.",
      "Construct approval state + expiry watcher at session start and subscribe them."
    ],
    "estimatedCompletionAfterCarefulMerge": "62-68%"
  }
```

Desktop Pack 16 done: 13 files (9 created, 4 modified). Picks up exactly where Pack 15 stopped: a fail-closed session-approval consent flow that enforces `requiresSessionApproval` before anything interactive turns on, a clipboard + file-transfer gate that mirrors the input gate (honoring emergency stop + expiry), and a session-expiry watcher that cuts control at the cap while leaving the screen view streaming.

The clean bit: `effectiveHostAccepted(hostAccepted, approval)` is the single bridge between Pack 15's host-accept flag and the consent requirement, so interactive features can't turn on until the device is trusted, the policy allows it, the host accepted, AND consent is satisfied. All three gates (input, clipboard/file, expiry) read the same permission set, so they can never disagree.